

National University of Singapore

School of Computing

CS3103 – Computer Networks Practice

LAB 2 - Introduction to mini-net

Due: 5th September, 2025 at 23:59

Mininet is a network emulator that allows you to create and test complex networks using virtual hosts, switches, and links. This exercise will introduce you to Mininet through a simple topology and a small design challenge.

Prerequisites:

- Ensure you have Mininet installed.
- Basic understanding of networking concepts such as hosts, switches, and links.

Objectives:

- Learn the basic commands in Mininet
- Learn how to create basic network topologies in Mininet
- Learn about Mininet API

Reference reading list –

- [1] Mininet, <http://mininet.org/>
- [2] OpenFlow WhitePaper , <http://archive.openflow.org/documents/openflow-wp- latest.pdf>
- [3] Floodlight Controller, <http://www.projectfloodlight.org/>

Deliverables –

[1] For this lab assignment, you are required to submit **only** one zip file containing the Python file that contains your custom topology, configuration and the constraints and another file containing screenshots of the outputs.

- **File format:** .zip with .py (Python script) and **word/pdf** file with relevant screenshots of output. Add comments where necessary to understand your code.
- Ensure that your file is named appropriately as: Name_MatricNumber.zip and contains all the code necessary to create and run the topology along with the relevant screenshots. To understand, add comments in the code where necessary.

Note: MAC users can either use the Lab PCs and follow the instruction set for window users as given below **OR** follow the lab instructions for MAC users below (which will be to download VMWare Fusion and Mininet Image, refer to details below):

Procedure -

Download the pre-requisites –

For Windows users -

- 1) Download the MININET Image :
<https://www.comp.nus.edu.sg/~bhojan/CS3103/SDN/Mininet-VM.zip>

2) Download and install a virtualization system VirtualBox (<https://www.virtualbox.org/wiki/Downloads>). It is free and works on OS X, Windows, and Linux.

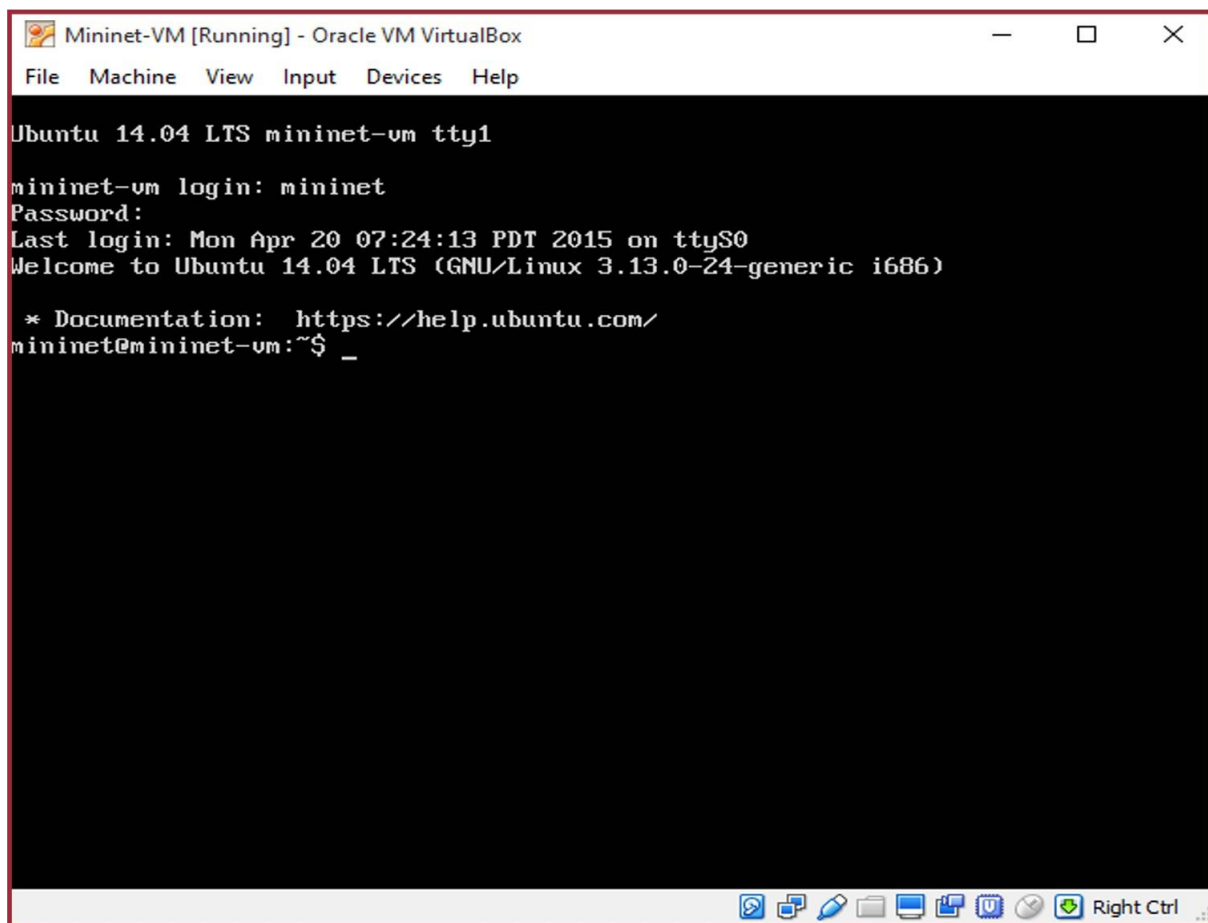
Starting the Mininet VM

3) Start VirtualBox.

4) Start the VM. Once it is started. Login using the following credentials:

Login: mininet

Password: mininet



5) Start the GUI environment by executing:

\$ startx

FOR MAC USERS:

Pre-requisites –

A) Download **VMWare Fusion 13.6.0** from the official website:

<https://support.broadcom.com/group/ecx/productdownloads?subfamily=VMware+Fusion>

B) Download the Mininet image from

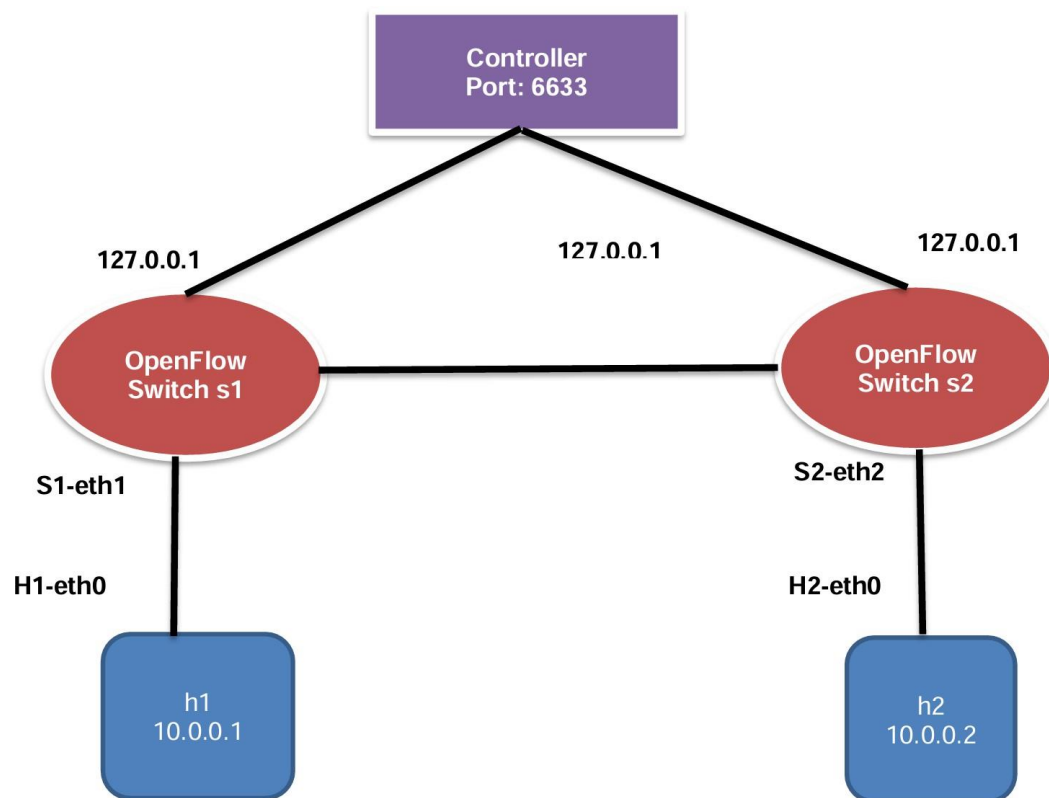
<https://www.comp.nus.edu.sg/~bhojan/CS3103/SDN/MininetMACarm.zip>

Starting the Mininet VM on VMWare Fusion

- 1) Open **VMWare Fusion**.
- 2) Import the **Mininet VM** into VMWare Fusion by selecting the downloaded image file.
- 3) Once the VM is imported, start the VM.
- 4) **Login credentials:**
 - Username: mininet
 - Password: mininet
- 5) *Note:* The Ubuntu Server VM already has the ubuntu-desktop package installed, so there is no need to run startx.

PART A) CREATING A VIRTUAL TOPOLOGY –

In this experiment, you will be creating a Linear Topology as shown in figure below:



- 6) Open a Terminal, you can view the code for creating a linear topology in

~/LinearTopo.py [The file is available here:
<https://www.comp.nus.edu.sg/~bhojan/CS3103/SDN/>]

- Topo: the base class for Mininet topologies
- addSwitch(): adds a switch to a topology and returns the switch name
- addHost(): adds a host to a topology and returns the host name
- addLink(): adds a bidirectional link to a topology (and returns a link key, but this is not important). Links in Mininet are bidirectional unless noted otherwise.
- Mininet: main class to create and manage a network
- start(): starts your network
- stop(): stops your network

7) Start Wireshark by executing “*sudo wireshark &*” in one of the terminals. Ignore any errors if displayed.

8) In Wireshark, select the interface as “**Loopback : Lo**” , and start capturing.

9) **FOR WINDOWS USERS:** Run the topology by executing the following commands:

```
$ chmod +x LinearTopo.py
```

```
$ sudo mn --custom LinearTopo.py --topo mytopo
```

NOTE FOR MAC USERS: Run the topology by executing the following commands:

```
$ chmod +x LinearTopo.py
```

```
$ sudo mn --custom LinearTopo.py --topo mytopo -v debug
```

This commands runs Mininet with your custom topology and the debug flag enabled (-v debug) to show detailed log information.

Verify that the topology is created as in the below screenshot:

```
mininet@mininet-vm:~$ sudo mn --custom LinearTopo.py --topo mytopo
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s3 s4
*** Adding links:
(h1, s3) (s3, s4) (s4, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
s3 s4 ...
*** Starting CLI:
mininet>
```

10) Check out the captures in Wireshark. You can focus only on Openflow messages by typing “of” as the filter. Analyze the captures.

11) Now that, the topology is created, we can try various things on the network. However, the first thing, we will measure the bandwidth between hosts h1, and h2. Run the following command to measure the bandwidth between the hosts :

```
$ iperf
```

Note for MAC Users – Once iperf has run its course, terminate it by pressing CTRL + C.

12) Now exit the Mininet, and remove the virtual network by typing the following command :

```
$ exit
```

Note for MAC Users for PART B and PART C: The remaining instructions for Part B and Part C remain largely unchanged for MAC users, with the only differences being the use of **VMWare Fusion** instead of VirtualBox and the debug flag added.

PART B) Creating a virtual topology with constraints:

Now, we will create a virtual network, with the following constraints:

a) Bandwidth = 10Mbps.

b) Delay = 5ms.

13) View the code for a constrained link network in ~/LinearTopoConstr.py. [The file is available here: <https://www.comp.nus.edu.sg/~bhojan/CS3103/SDN/>]

Note the difference in the new topology by checking the line, which creates the link between the switches. Note the parameters given for the link.

14) Run the topology by executing the following commands:

```
$ chmod +x LinearTopoConstr.py
```

```
$ sudo mn --custom LinearTopoConstr.py -- topo mytopo --link tc
```

15) Now, we can verify the total bandwidth by running iperf again.

```
$ iperf
```

Note for MAC Users – Once iperf has run its course, terminate it by pressing CTRL + C.

16) Now exit the Mininet, and remove the virtual network by typing the following command :

```
$ exit
```

PART C) Network Design Challenge for custom topology –

Scenario – Industry requirement

You are working as a network engineer for a small company with two separate departments that need to be connected via switches. Each department has multiple hosts. Your goal is to create a basic topology that reflects this scenario:

- Department A has 2 hosts (h1, h2).
- Department B has 2 hosts (h3, h4).

- Each department's hosts are connected to a local switch (s1 for Dept A, s2 for Dept B).
- The two switches are connected to each other.

Task 1: Build and Test Your Network –

- a) Design the topology
- b) Modify the code to reflect the topology in the LinearTopo.py file.
- c) Run the topology and test the connectivity. Ensure hosts in different departments can communicate through the switches.

Task 2: Add the following constraints to your Network and modify the code file –

- a) Add the following bandwidth and delay constraints:
bw = 100Mbps, delay = 10ms
